

从已验证能力到合法派生

面向 Agent 的预研开发形式方法

(教材式草案 · 持续修订)

ascendc 工程 · 专题分支 `research/formal-lang-dag`

摘要

愿景：把「层层递进、先验证底层再拼高层」的工程直觉，提升为一套可教学、可门禁、可对照实验的形式方法——使 Agent（以及人类协作者）在复杂预研任务中，**只沿已验证能力及其合法组合**生长实现，从机制上压缩跳层发明与幻觉空间；最终服务一般预研开发，而非仅服务某一密码标准的算子清单。

写法：仿教材而非论文——先从特例讲故事，再抽象数学对象，再复盘与前瞻；证明与完备性后置，允许边做边补。**地位：**`docs/research/` 调研草稿，非定稿、非交付验收依据。

压力测试床：ML-KEM-1024 链上的 `pke.keygen / kem.keygen / kem.encaps`，并以 `kem.decaps` 为理论前瞻试金石；`*-correctness-*` 作为「只求正确、不计优化」的对照组标本。

目录

1 愿景、读者与本文怎么读	2
1.1 愿景（一句话）	2
1.2 我们希望最终得到什么	2
1.3 研究路线：从特殊到一般	2
1.4 阅读地图（教材顺序）	2
2 一个完整例子：我们怎样做出 ML-KEM-1024 的 KEM.KeyGen	2
2.1 我们要交付什么	3
2.2 FIPS 203 原文：三层算法怎样套在一起	3
2.3 动手之前，我们手里已经有什么	5
2.4 我们怎样探出来：工程路径与数据依赖	6
2.5 从这个例子出发，我们应当问什么？	7
2.6 本工程现在能回答这四组问题吗？	8
3 方法论所需的数学背景	9
3.1 本章范围、记号与阅读约定	9
3.2 集合、笛卡尔积、关系与函数	10
3.3 有向图、路与环	11
3.4 有向无环图与拓扑序	12
3.5 闭包：从一般概念到依赖闭包	13

3.5.1	在运算下封闭	13
3.5.2	闭包：最小的封闭扩张	13
3.5.3	闭包算子	14
3.5.4	关系的传递闭包：闭包的第一个正式例子	15
3.5.5	有向图上的依赖闭包	16
3.5.6	依赖闭包的迭代、算法与算子性质	17
3.5.7	目标集的闭包、基与缺项	17
3.6	三层对象：库、一次派生、过程	20
3.7	字母表、字与语言	21
3.8	文法、产生式、派生与派生树	22
3.9	成员判定、禁字与组合不免费	23
3.10	迁移系统、配置与接受	24
3.11	证书的非单调更新	25
3.12	回扣第 2 章四组问题	25
3.13	本章小结与下一章预告	25
4	从数学到工程对象：节点、边与证书	25
4.1	三个层面（复习）	26
4.2	节点（已验证能力）	26
4.3	边的三种类型	26
4.4	证书状态（含非单调）	26
4.5	仓库机制与数学对象的对照（直觉）	27
4.6	立论（承接第 3 章）	27
5	门禁与工作流：Agent 被允许做什么	27
5.1	合法性规则（v0）	27
5.2	四步工作流	27
5.3	域无关性	27
6	复盘：理论如何对照 kem.keygen 与 kem.encaps	28
6.1	复盘的目的	28
6.2	kem.keygen 校准问题单	28
6.3	kem.encaps 复盘要点（讨论共识）	28
6.4	期望产出的「具象化对照表」（后续填空）	28
7	前瞻：用理论指导尚未完成的 kem.decaps	28
7.1	为什么 Decaps 适合做试金石	28
7.2	前瞻时 Agent（或人类）应交付的最小闭包	29
7.3	成功标准（阶段性）	29
8	对照组：correctness 标本的意义	29
8.1	为何保留 correctness	29

8.2	在方法论中的角色	29
8.3	建议比较轴	29
9	未决议事项、日程与维护	30
9.1	刻意未决议	30
9.2	近期填空顺序（与「特殊 → 一般」一致）	30
9.3	文档与分支约定	30
9.4	编译	30

1 愿景、读者与本文怎么读

1.1 愿景（一句话）

我们想表达的是：预研开发**不应当**变成「对着算法说明书自由发挥写核」。我们应当只在**已经拿到证书**的能力之上，按规则组合出更复杂的任务；静态 DAG 只是「引理库谁依赖谁」的一张投影图，真正的作业过程还要另说。

1.2 我们希望最终得到什么

1. 一套**教材式叙述**：新人 / 新 Agent 能从特例读懂「图从哪来、树怎么长」；
2. 一套**可操作门禁**：任务启动必须先交依赖闭包与缺项，再写码；
3. 一套**对照实验协议**：方法论指导下的实现 vs correctness 标本 vs 人工指导路径；
4. (成型后) 可解释的接受/拒绝轨迹，以及必要的证明——**不作为本稿前置条件**。

1.3 研究路线：从特殊到一般

阶段	做什么	刻意后置
特例引出	工程故事 → 图与语法树	完备公式
套公式	用数学 复述 已见现象	证明
改公式	缺口反过来改理论	为漂亮删工程事实
套别例	Encaps 复盘；换域；Decaps 前瞻	一次塞太多缺口
迭代方法论	门禁与流程收敛	急进 Rule/Skill
成型后	证明、可解释性	过早锁死抽象

1.4 阅读地图（教材顺序）

1. 第 2 章：**完整例子**（KEM.KeyGen：算法原文、我们有什么、怎样做出来、引出课题）；
2. 第 3 章：**方法论数学背景**（图、闭包、形式语言、自动机直觉——**不含密码/NTT 代数**）；
3. 第 4–5 章：**装进工程对象与门禁**（节点/边/证书；Agent 工作流）；
4. 第 6–7 章：**复盘与前瞻**（用理论看实现）；
5. 第 8 章：**correctness 对照组**；
6. 第 9 章：未决议事项与维护约定。

讨论过程的流水账见同目录 形式语言 DAG 预研讨论纪要.md（单文件持续刷新）。

2 一个完整例子：我们怎样做出 ML-KEM-1024 的 KEM.KeyGen

本章用一个**已经做完**的真实任务当例子。我们先说清楚「要算什么」，再贴 FIPS 203 里的算法原文，然后交代我们手里已经有什么、前期做过哪些实验、最后怎样拼出 KEM.KeyGen。例子讲完之后，我们再**提问**——这些问题应当能把读者自然带到后文的研究课题里。

2.1 我们要交付什么

我们的目标是：在 Ascend AI Core 上用 AscendC 实现 **FIPS 203 ML-KEM-1024 的密钥生成** (ML-KEM.KeyGen, 即 Algorithm 19)。

读者可以把交付物理解成下面这张黑盒表 (细节以 stable 目录 customspect 为准)：

方向	内容	说明
输入	seed_d.bin (4B)	Host 写入；设备侧派生 d 、 z
输入	NTT LUT (静态)	与 PKE KeyGen 同表
输出	ek_kem.bin (1568 B)	与 ek_PKE 相同
输出	dk_kem.bin (3168 B)	与 liboqs 展开布局一致

我们不把 d 、 z 、中间 $\hat{A}$ 、 $\hat{s}$ 等当作交付文件落盘。验收方式是：run.sh 跑 CPU 孪生 + SIM，输出与 golden / liboqs 对拍一致。

2.2 FIPS 203 原文：三层算法怎样套在一起

标准把 KEM 密钥生成拆成三层。下面给出与 **FIPS 203 原文一致的参数、Input/Output 与步骤编号**。正文引用 NIST FIPS 203 Algorithm 13 / 16 / 19；本文实例固定为 ml_kem_1024 ($k=4$ 时 $|ek|=1568$ B, $|dk|=3168$ B)。

Algorithm 19 ML-KEM.KeyGen() (FIPS 203 §7.1)

参数集 (ml_kem_1024): $n=256$, $q=3329$, $k=4$, $\eta_1=\eta_2=2$, $d_u=11$, $d_v=5$ 。

Input: (无显式输入； d 、 z 在算法内由 RBG 采样，见步骤 1-2。)

Output: 封装密钥 $ek \in \mathbb{B}^{384k+32}$ ；

解封密钥 $dk \in \mathbb{B}^{768k+96}$ 。

1: $d \leftarrow \text{\$B}^{32}$

2: $z \leftarrow \text{\$B}^{32}$

3: if $d = \text{NULL}$ or $z = \text{NULL}$ then

4: return $\perp$

RBG 失败指示

5: end if

6: $(ek, dk) \leftarrow \text{ML-KEM.KeyGen_internal}(d, z)$

7: return (ek, dk)

Algorithm 16 ML-KEM.KeyGen_internal(d, z) (FIPS 203 §6.1)

参数集 (ml_kem_1024): $n=256$, $q=3329$, $k=4$, $\eta_1=\eta_2=2$, $d_u=11$, $d_v=5$ 。

Input: 随机种子 $d \in \mathbb{B}^{32}$ ；随机种子 $z \in \mathbb{B}^{32}$ 。

Output: 封装密钥 $ek \in \mathbb{B}^{384k+32}$ ；

解封密钥 $dk \in \mathbb{B}^{768k+96}$ 。

1: $(ek_{\text{PKE}}, dk_{\text{PKE}}) \leftarrow \text{K-PKE.KeyGen}(d)$

2: $ek \leftarrow ek_{\text{PKE}}$

3: $dk \leftarrow dk_{\text{PKE}} \| ek \| H(ek) \| z$

4: return (ek, dk)

其中 $H = \text{SHA3-256}$ (FIPS 203 记号 H)。

Algorithm 13 K-PKE.KeyGen(d) (FIPS 203 §5.1)

参数集 (ml_kem_1024): $n=256$, $q=3329$, $k=4$, $\eta_1=\eta_2=2$, $d_u=11$, $d_v=5$ 。

Input: 随机种子 $d \in \mathbb{B}^{32}$ 。

Output: 加密密钥 $ek_{\text{PKE}} \in \mathbb{B}^{384k+32}$;

解密密钥 $dk_{\text{PKE}} \in \mathbb{B}^{384k}$ 。

```

1:  $(\rho, \sigma) \leftarrow G(d \| k)$  域分离: 字节 33 为模块维  $k$ 
2:  $N \leftarrow 0$ 
3: for  $i \leftarrow 0$  to  $k - 1$  do
4:   for  $j \leftarrow 0$  to  $k - 1$  do
5:      $\hat{A}[i, j] \leftarrow \text{SampleNTT}(\rho \| j \| i)$ 
6:   end for
7: end for
8: for  $i \leftarrow 0$  to  $k - 1$  do
9:    $s[i] \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(\sigma, N));$   $N \leftarrow N + 1$ 
10: end for
11: for  $i \leftarrow 0$  to  $k - 1$  do
12:    $e[i] \leftarrow \text{SamplePolyCBD}_{\eta_1}(\text{PRF}_{\eta_1}(\sigma, N));$   $N \leftarrow N + 1$ 
13: end for
14:  $\hat{s} \leftarrow \text{NTT}(s)$ 
15:  $\hat{e} \leftarrow \text{NTT}(e)$ 
16:  $\hat{t} \leftarrow \hat{A} \circ \hat{s} + \hat{e}$  NTT 域矩阵-向量乘加
17:  $ek_{\text{PKE}} \leftarrow \text{ByteEncode}_{12}(\hat{t}) \| \rho$ 
18:  $dk_{\text{PKE}} \leftarrow \text{ByteEncode}_{12}(\hat{s})$ 
19: return ( $ek_{\text{PKE}}, dk_{\text{PKE}}$ )

```

读者只需抓住一条结构链: $\text{Alg.19} \Rightarrow \text{Alg.16} \Rightarrow \text{Alg.13}$ 。Alg.19 比 Alg.13 多出来的主要是步骤 1-2 的 z 、Alg.16 步骤 3 的拼接——并不是把 NTT 或矩阵乘从头再写一遍。

本仓工程差异 (验收已锁定, 写在算法旁便于对照)

- 我们可复现跑分时, Host 只提供 4 B `seed_d`; d 与 z 在设备 UB 内派生 (对应 Alg.19 步骤 1-2 的确定性扩展)。
- 我们对拍 liboqs 时, dk 为 3168 B 展开式 $dk_pke \| ek \| H \| z$, 与 FIPS 代数最小形态一致, 但字节布局与偏移以 `customspec` 为准。

2.3 动手之前，我们手里已经有什么

到做 KEM.KeyGen 之前，本仓库并不是从一张白纸开始。我们已经在 AscendC 上做过大量单点与分段实验，并且多数有 CPU+SIM 对拍记录。下面这张**资产清单**把「当时手里有什么」写清楚；文字说明与表格对照阅读。

四类家底（文字）

1. **平台层 (P0)**：我们验证过 DataCopy、TPipe/TQue、MIX 核同步、MMAD tiling 等。
2. **密码原语层 (P1)**：我们验证过 NTT (poly-batch)、basemul、SHAKE/SHA3、CBD、ByteEncode₁₂、SampleNTT 等。
3. **PKE 全链 (P2)**：我们已有交付级 stable PKE KeyGen (2 launch, KAT 通过)。
4. **对照标本**：我们保留 `*-correctness-*`，只求可执行与字节正确，不计优化。

资产清单（表格；锚点以活跃 INDEX 为准）

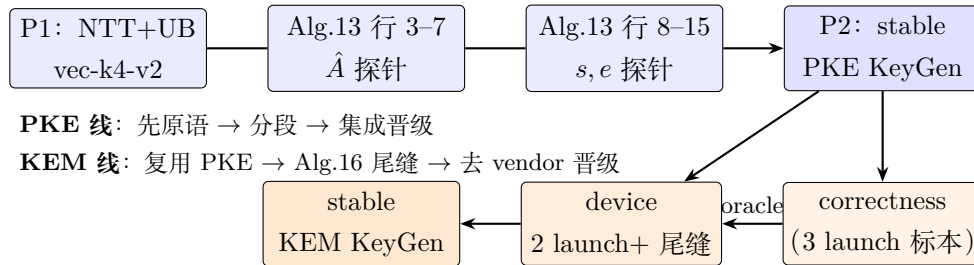
层	能力（简述）	仓库锚点（示例）	证书	备注
P0	DataCopy / 搬运	docs/notes/ascendc-DataCopy ...	文档 + 探针	平台知识库
P0	MMAD + tiling	pass-int8-matmul-cube-...等	CPU+SIM	闭合条件见 notes
P0	TPipe / 细同步	KeyGen prep 技术总结	CPU+SIM	翻车高发区
P1	NTT 三段式 poly-batch	pass-fix-...-vec-k4-v2	CPU+SIM	ML-KEM 活跃基线
P1	SampleNTT / Alg.7	pass-fix-f203-alg7-sample-ntt-cpu	CPU+SIM	
P1	CBD $\eta=2$	pass-fix-f203-alg8-cbd-eta2-k4	CPU+SIM	
P1	ByteEncode ₁₂	pass-fix-... -byteencode12-vec-k4	CPU+SIM	
P2	Alg.13 行 3-7 $\hat{A}$	pass-fix-f203-alg13-lines3-7-cpu	CPU+SIM	≈542k tick KEM 直接依赖
P2	Alg.13 行 8-15 s, e	pass-fix-f203-alg13-lines8-15-cpu	CPU+SIM	
P2	Alg.13 device 全链	pass-fix-f203-alg13-device-keygen-k4	CPU+SIM	
P2	stable PKE KeyGen	stable-fips203-mlkem-pke-keygen-k4	交付 KAT	
P3	KEM KeyGen correctness	fix-f203-alg19-kem-keygen-correctness-k4	CPU+SIM	对照标本
P3	KEM KeyGen device	pass-fix-f203-alg19-kem-keygen-device-k4	CPU+SIM	≈713k tick
P3	stable KEM KeyGen	stable-fips203-mlkem-kem-keygen-k4	交付 KAT	本例终点

当我们决定做 KEM.KeyGen 时，我们并不是让 Agent 从 FIPS 伪代码一行行翻译成 AscendC，而是心里已经有上表这张「哪些积木是绿的」的清单。

2.4 我们怎样探出来：工程路径与数据依赖

这张清单不是一天长出来的。本节用两张图把同一件事说清楚：**第一张**是仓库时间线（先长 PKE，再接 KEM 尾缝）；**第二张**是 FIPS Alg.19→16→13 的数据依赖（与 §2.2 步骤编号对照）。

图一：工程探路（PKE 线 → KEM 线）



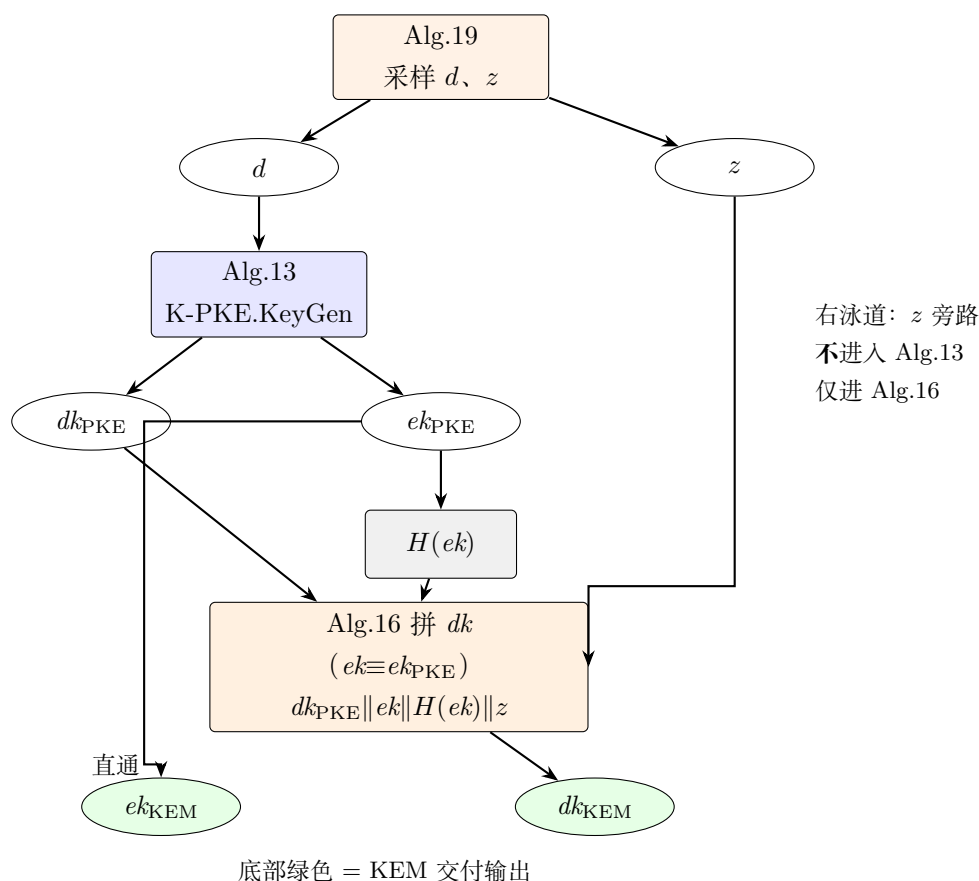
- **PKE 线：**我们先把 NTT、内积等做成向量探针；再把 Alg.13 拆成 $\hat{A}$ 、 s, e 、device 全链，逐段 PASS；最后集成晋级 stable。
- **KEM 线：**我们语义上调用 Alg.13（步骤 1 即 K-PKE.KeyGen）；设备上在 Launch-2 尾部追加 Alg.16 步骤 2-3（ H 、拼 dk ）；编排上保持 2-launch；验收上 correctness 作 oracle，device 去 vendor 后晋级 stable。

设备侧 Launch 与 FIPS 行的对应（本仓实现）

Launch	设备段	FIPS 行	要点
L1	f203_keygen_prep	Alg.13 步骤 1-15	双 AIV； $\hat{A}$ 、 $\hat{s}$ 、 $\hat{e}$
L2	mmad_custom + 尾缝	Alg.13 步骤 16-21	NTT、内积、ByteEncode
L2 尾	KemKgTailFused	Alg.16 步骤 2-3	z 、 $H(ek)$ 、写 dk

若用一句话概括这次成功：**我们把 FIPS 的三层算法，映射成「已证 PKE 链 + 一条单独想清楚的 KEM 尾缝」**，而不是在 KEM 任务里重写 NTT。

图二：FIPS 三层的数据依赖（图感） 图一讲「仓库里怎样长出来」；图二只看标准数据流——重计算在 Alg.13， z 不进 Alg.13，只进 Alg.16 的 dk 拼接； ek_{KEM} 对 ek_{PKE} 是直通。



读图时抓住三点：

1. **PKE 主体**：几乎全部重计算落在 Alg.13； d 进、 ek_{PKE}/dk_{PKE} 出。
2. **KEM 尾缝**：Alg.16 把已有片段拼成 dk_{KEM} ； ek_{KEM} 对 ek_{PKE} 直通（图底左侧绿叶），不重做 NTT。
3. **汇点而非树**：NTT、CBD、MMAD 等还不画进本图——它们也不专属于 KeyGen，Encrypt / Encaps 会复用；因此本图只是更大有向无环图里的一个汇点，不是一棵只往下长的树。

2.5 从这个例子出发，我们应当问什么？

例子讲完，真正重要的是**问题**。如果我们的研究课题是「怎样让 Agent 预研少幻觉、少撞墙」，那么面对 KEM.KeyGen 这个故事，我们至少应当认真问下面四组问题。

第一组：标准与工程之间的缝隙

1. FIPS 只告诉我们算什么；它并没有告诉我们，在 Ascend 上应当先验哪儿块、按什么顺序拼。我们凭什么在动手前就说「可以直接站在 stable PKE 上」——凭的是目录名，还是某种可写的证书？
2. 探针列表 (INDEX) 像一张「能力菜单」。菜单本身是否等于**合法拼装说明书**？若不等，缺的那一页应当长什么样？

第二组：「单点都对」之后，组合还对吗

3. 当 NTT 探针 PASS、PKE KeyGen PASS 之后，**谁**来保证「2-launch 接缝」「Alg.16 尾拼 *dk*」也一定 PASS？我们能否把这种「组合不免费」写进理论，而不是每次靠工程师的经验？
4. 若我们把 correctness 路线（更多 launch、更保守同步）与 device/stable 路线（更少 launch、吃到优化）都称为「正确」，**我们究竟在验收什么**——仅仅是输出字节，还是也要验收「沿已证能力生长」？

第三组：Agent 若只知道算法和菜单，会发生什么

5. 若我们只给 Agent 一段 Alg.19 伪代码 + INDEX 里有哪些探针 PASS，**但不给**确定的拼装路线，搜索空间会有多大？我们亲眼见过：过程高度混乱、大量回退、极耗 token，最后只留下「能跑对」的 correctness 标本。
6. 我们能否把搜索空间收束为「只允许沿已证书边生长」，同时又**不把**工程优化堵死？收束的规则应当写在图里，还是写在某种「合法句子」里？

第四组：怎样把一次成功变成可复用的方法

7. 这次 KEM.KeyGen 的成功，能否被复述成一张**闭包表**：依赖谁、缺了谁、哪条边有证书、哪条边被禁止？若下一个人（或 Agent）做 Encaps / Decaps，我们能否**先交这张表再写码**？
8. 当上游改了 tiling 或同步壳，下游哪些节点应当自动标「脏」？我们是否需要一种比「记得上次 PASS」更可靠的失效传播？

2.6 本工程现在能回答这四组问题吗？

四组问题都很好，但「能回答」要分层次：工程实践里我们已有碎片，形式化方法论里我们尚未闭环。下表诚实对照（2026-07 专题讨论时的判断）。

问题组	本工程 已能 说明的	仍缺的	判断
第一组	INDEX、customspec、baseline-registry 已在约束引用来源	机器可读的「证书对象」；菜单 ≠ 拼装说明书的显式规则	部分能答
第二组	我们有 seam 探针/集成探针实践；correctness vs device 在 customspec 有对照	「组合不免费」未写成统一理论；验收口径仍以 I/O 为主	部分能答
第三组	correctness 历史是 反面教材 （高成本、不可控）	正向的「合法派生」门禁尚未工具化；Agent 仍可能绕开	能答负例；正例在建
第四组	单任务可手写 INTEGRATION_PLAN、闭包直觉	统一闭包表 schema；dirty 传播无自动机制	部分能答

逐组一句话

- **第一组**：我们能指着 stable PKE 说「KEM 应站在这里」，但还缺一张人人（含 Agent）都认的证书页。
- **第二组**：我们知道接缝要单独验（例如 2-launch、尾缝），但还没把这条经验收成**可检查的规则**。
- **第三组**：我们能用 correctness 证明「只给菜单会爆炸」；方法论要证明的是「给规则后能收敛」——这正是本专题要做的事。
- **第四组**：我们能复盘 KEM.KeyGen 的故事，但 Encaps/Decaps 还不能**先交闭包表再写码**作为硬门禁。

因此：**本工程能支撑这四组问题的提出与部分实证，但不能宣称方法论已经成立**。后文的工作，就是把表中「仍缺的」一列逐项补上，并用 Encaps / Decaps 做检验。

这些问题的共同指向 上面这些问题，表面上各不相同，但它们指向同一条研究主线：

我们能否把「已验证能力 + 合法组合 + 动态证书」整理成一套**可教、可检查、可对照实验**的形式方法——
让 Agent 的工作从「在菜单里瞎拼」变成「在自动机上走合法派生」？

后文先补**方法论所需的数学背景**（第 3 章：图、闭包、语言、自动机），再把这些对象**装进工程语境**并写成门禁（第 4-5 章），最后用 Encaps 复盘、Decaps 前瞻检验这套语言是否管用。**本章与后文均不展开 PQC / 数论变换等密码代数**——那些细节见 docs/notes/；本专题专注方法论。

3 方法论所需的数学背景

第 2 章给出了工程故事与问题感。本章的任务是：把后续方法论要用到的数学，按教材体例由**浅入深**写清楚——给出编号的定义、命题、定理、推论与引理，并固定双例节奏：**先举一个纯数学小例子，再举第 2 章 KEM.KeyGen 的工程例子**。讲述顺序上：集合与关系 → 图与 DAG 之后，**先建立一般闭包**（封闭性、最小封闭扩张、闭包算子），再讲关系的传递闭包，最后才是依赖闭包与闭包表。本章**不讨论格、MLWE、NTT 等密码代数**。

3.1 本章范围、记号与阅读约定

	本章建设	本章明确不建设
对象	依赖、闭包、合法语言、配置转移、证书演化	密码方案正确性证明
体例	定义 / 命题 / 定理 / 引理 / 推论 + 双例	百科式堆砌无证明的名词
深度	以「后文门禁够用」为准；标准结果给证明梗概	可判定性、复杂度专论（后置）

注 3.1（阅读目标）. 读完本章后，应能严格区分并陈述：(i) 能力 DAG 回答什么；(ii) 一次派生 / 语法树回答什么；(iii) 配置转移如何描述接受与拒绝。

3.2 集合、笛卡尔积、关系与函数

定义 3.1 (集合与属于关系). 称由确定对象组成的汇集为**集合**。若对象 x 属于集合 A , 记 $x \in A$; 否则记 $x \notin A$ 。不含任何元素的集合称为**空集**, 记作 $\emptyset$ 。若 A 的每个元素都属于 B , 称 A 为 B 的**子集**, 记 $A \subseteq B$ 。

定义 3.2 (幂集). 设 V 为集合。由 V 的**全部子集**组成的集合称为 V 的**幂集**, 记作 2^V (亦常记作 $\mathcal{P}(V)$)。用集合造集的写法即

$$2^V = \{A \mid A \subseteq V\}.$$

特别地: $\emptyset \in 2^V$, $V \in 2^V$; 若 V 有限且 $|V| = n$, 则 $|2^V| = 2^n$ (这正是记号 2^V 的来源)。

映射 $f: 2^V \rightarrow 2^V$ 表示: 输入是 V 的某个子集, 输出仍是 V 的某个子集。

注 3.2 (为何是 2^V , 而不是 3^V 、 4^V). 底数 2 来自构造子集时的**二元选择**: 对 V 中每个元素, 只有「放进该子集」或「不放进」两种可能。因此不同子集的个数是 $2 \times 2 \times \cdots \times 2$ (共 $|V|$ 个因子), 即 $2^{|V|}$ 。若改写成 3^V , 一般既不等于「全部子集的集合」, 也没有上述计数意义 (除非另作约定, 例如讨论「每个元素三种染色」的函数集 $\{0, 1, 2\}^V$, 那是另一对象)。

等价观点: 把 2 看成二元集 $\{0, 1\}$, 则 2^V 也可理解为「全体函数 $V \rightarrow \{0, 1\}$ 」(特征函数: 取 1 表示属于子集, 取 0 表示不属于); 这与「全部子集」一一对应。

例 3.1 (纯数学: 幂集). 令 $V = \{a, b\}$ 。则

$$2^V = \{\emptyset, \{a\}, \{b\}, \{a, b\}\}, \quad |2^V| = 4 = 2^2.$$

定义 3.3 (笛卡尔积). 设 A, B 为集合。其**笛卡尔积**为

$$A \times B = \{(a, b) \mid a \in A, b \in B\}.$$

元素 (a, b) 称为**有序对**; 一般地 $(a, b) = (a', b')$ 当且仅当 $a = a'$ 且 $b = b'$ 。

定义 3.4 (二元关系). 设 A, B 为集合。若 $R \subseteq A \times B$, 则称 R 为从 A 到 B 的**二元关系**。当 $A = B$ 时, 也称 R 为 A 上的二元关系。若 $(a, b) \in R$, 常简记为 $a R b$ 。

定义 3.5 (关系的定义域与值域). 对关系 $R \subseteq A \times B$, 定义

$$\text{dom}(R) = \{a \in A \mid \exists b \in B, (a, b) \in R\}, \quad \text{ran}(R) = \{b \in B \mid \exists a \in A, (a, b) \in R\}.$$

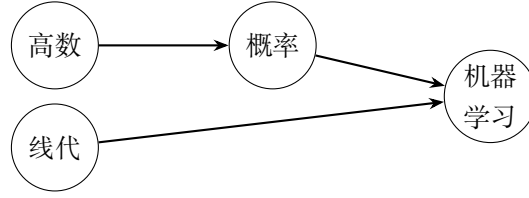
定义 3.6 (函数). 设 A, B 为集合。称关系 $f \subseteq A \times B$ 为从 A 到 B 的**函数** (记 $f: A \rightarrow B$), 若对每个 $a \in A$, 存在唯一的 $b \in B$ 使得 $(a, b) \in f$ 。此时记 $b = f(a)$, 称 b 为 a 在 f 下的**像**。

性质 3.1 (关系与函数的对比). 二元关系允许「一个 a 对应多个 b 」; 函数禁止这一点。后文「证书状态赋值」按函数理解; 「依赖边」按一般关系理解。

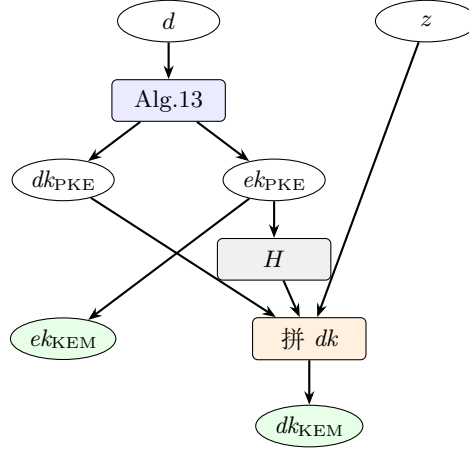
例 3.2 (纯数学: 课程先修关系). 令 $A = \{\text{高数}, \text{线代}, \text{概率}, \text{机器学习}\}$, 定义 $R \subseteq A \times A$ 为

$$R = \{(\text{高数}, \text{概率}), (\text{线代}, \text{机器学习}), (\text{概率}, \text{机器学习})\}.$$

则 $(\text{高数}, \text{概率}) \in R$, 但 $(\text{概率}, \text{高数}) \notin R$ (有序性)。映射 $\text{grade}: A \rightarrow \{A, B, C\}$ 若存在, 则是函数: 每门课恰一个成绩。



例 3.3 (工程: §2 数据依赖中的关系). 令 $V_0 = \{d, z, \text{Alg.13}, ek_{\text{PKE}}, dk_{\text{PKE}}, H, ek_{\text{KEM}}, dk_{\text{KEM}}\}$. §2 图二中的箭头构成 V_0 上的关系 R_{KG} , 例如 $(d, \text{Alg.13}) \in R_{\text{KG}}$, $(z, \text{Alg.13}) \notin R_{\text{KG}}$. 「把 ek_{PKE} 直通记为 ek_{KEM} 」可视为部分函数 $\iota: \{ek_{\text{PKE}}\} \rightarrow \{ek_{\text{KEM}}\}$. 下图为关系 R_{KG} 的示意 (数据流语义; 闭包节将改用依赖方向).



3.3 有向图、路与环

定义 3.7 (有向图). 有向图是一个有序对 $G = (V, E)$, 其中 V 为非空有限集 (称为**顶点集**), $E \subseteq V \times V$ 为**有向边集**. 若 $(u, v) \in E$, 称有向边 $u \rightarrow v$, 并称 u 为该边的**起点**, v 为**终点**.

定义 3.8 (出邻域与入邻域). 对 $v \in V$, 定义

$$N^+(v) = \{u \in V \mid (v, u) \in E\}, \quad N^-(v) = \{u \in V \mid (u, v) \in E\}.$$

分别称为 v 的**出邻域**与**入邻域**; $|N^+(v)|$ 、 $|N^-(v)|$ 称为**出度**、**入度**.

定义 3.9 (有向途径、路径与环). 设 $G = (V, E)$.

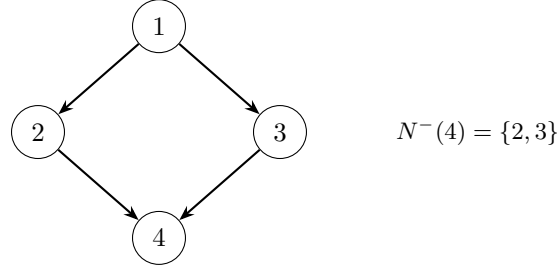
1. 顶点序列 $v_0, v_1, \dots, v_k$ ($k \geq 0$) 称为**有向途径**, 若对所有 $0 \leq i < k$ 有 $(v_i, v_{i+1}) \in E$; 称 k 为该途径的**长度**.
2. 若途径中顶点两两不同, 则称为**有向路径** (简称**路径**).
3. 若 $k \geq 1$ 、 $v_0 = v_k$, 且 $v_0, \dots, v_{k-1}$ 两两不同, 则称为**有向环** (简称**环**).

定义 3.10 (可达). 若存在从 u 到 v 的有向途径, 则称 v 从 u **可达**, 记 $u \rightsquigarrow_G v$ (在上下文明确时可略去 G). 约定任意 v 满足 $v \rightsquigarrow v$ (长度为 0 的途径).

引理 3.1 (路径与途径). 在有限有向图中, $u \rightsquigarrow v$ 当且仅当存在从 u 到 v 的有向路径.

证明. 若存在途径, 删去其上环段可得路径; 反之, 路径本身是途径. □

例 3.4 (纯数学：四点有向图). 令 $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (1, 3), (2, 4), (3, 4)\}$. 则 $1 \rightsquigarrow 4$, 且存在两条不同路径 $1 \rightarrow 2 \rightarrow 4$ 与 $1 \rightarrow 3 \rightarrow 4$. 4 的入邻域为 $N^-(4) = \{2, 3\}$, 出邻域 $N^+(4) = \emptyset$. 该图无环。



例 3.5 (工程：KeyGen 数据流图是有向图). §2 图二给出顶点 (如 d 、 z 、Alg.13、 dk_{KEM}) 与有向边。 dk_{KEM} 的入度大于 1 (多源汇合); $z \rightsquigarrow dk_{\text{KEM}}$ 但 $z \not\rightsquigarrow \text{Alg.13}$. 这与「树只有单父」不同：有向图允许多父汇点 (见上节工程例图中「拼 dk 」结点)。

3.4 有向无环图与拓扑序

定义 3.11 (DAG). 若有向图 G 不含有向环, 则称 G 为**有向无环图** (Directed Acyclic Graph, DAG)。

定义 3.12 (拓扑序). 设 $G = (V, E)$, $|V| = n$. 顶点序列 $v_1, \dots, v_n$ 称为 G 的一个**拓扑序**, 若它是 V 的排列, 且对每条边 $(v_i, v_j) \in E$ 都有 $i < j$ 。

定理 3.1 (DAG 与拓扑序的刻画). 设 G 为有限有向图. 则下列条件等价:

1. G 是 DAG;
2. G 存在拓扑序。

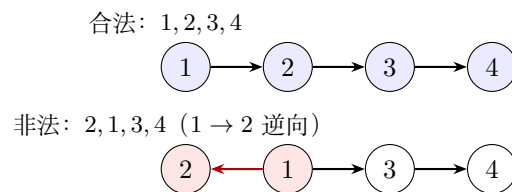
证明梗概. (2) $\Rightarrow$ (1): 若有环, 则环上边无法全部满足「起点下标小于终点下标」。

(1) $\Rightarrow$ (2): DAG 中必有入度为 0 的顶点; 取其为 v_1 , 删去后再对剩余图归纳。 $\square$

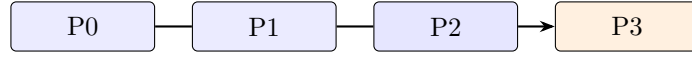
推论 3.1 (可按先决条件调度). 若依赖关系构成 DAG, 则存在线性顺序, 使得每条依赖边都从「先处理」指向「后处理」。

注 3.3 (预研建模提示). 「先验证下层能力, 再拼上层任务」要求依赖关系可拓扑排序, 因而应建模为 DAG. 若出现环, 通常意味着节点切分不清或误把双向约束画成了依赖。

例 3.6 (纯数学：给四点图写拓扑序). 续前例 $E = \{(1, 2), (1, 3), (2, 4), (3, 4)\}$. $1, 2, 3, 4$ 与 $1, 3, 2, 4$ 都是拓扑序; $2, 1, 3, 4$ 不是 (边 $1 \rightarrow 2$ 逆向)。



例 3.7 (工程：能力依赖链). 将「P0 平台事实 $\rightarrow$ P1 原语 $\rightarrow$ P2 PKE.KeyGen $\rightarrow$ P3 KEM.KeyGen」视为一条链, 则它是 DAG, 且该顺序本身就是拓扑序. 真实仓库图会有多父多子, 但仍应保持无环。



注 3.4 (DAG 不回答的问题). 能力 DAG 描述**引理库中的静态依赖投影** (谁可以依赖谁)。它不自动给出「某次做 KEM.KeyGen 时 Agent 实际展开的那棵树」——后者是派生 (见后文)。

3.5 闭包：从一般概念到依赖闭包

前几节已有集合、关系、有向图。本节先回答一个更基本的问题：**数学里说的「闭包」究竟是什么？**然后再说明：后文的「依赖闭包 / 闭包表」，只是同一种想法在能力 DAG 上的具体化。

3.5.1 在运算下封闭

定义 3.13 (一元运算下的封闭性). 设 U 为全集 (讨论所在的大集合), $A \subseteq U$, 又设 $\varphi: U \rightarrow U$ 为一元运算 (函数)。称 A 对 φ **封闭** (或「在 φ 下封闭」), 若

$$\forall x \in A, \quad \varphi(x) \in A.$$

即：从 A 里任取元素做一次 φ , 结果仍落在 A 里。

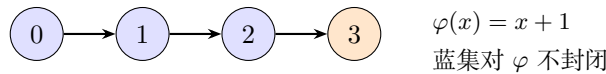
定义 3.14 (二元运算下的封闭性). 设 $*$: $U \times U \rightarrow U$ 为二元运算。称 $A \subseteq U$ 对 $*$ **封闭**, 若

$$\forall x, y \in A, \quad x * y \in A.$$

定义 3.15 (对一族运算封闭). 若给定若干运算 (可混一元、二元), 称 A 对它们**封闭**, 当且仅当 A 对其中每一个运算都封闭。

例 3.8 (纯数学：封闭与不封闭). 取 $U = \mathbb{Z}$ 。

- 偶数集 $2\mathbb{Z}$ 对加法封闭：两偶数之和仍为偶数。
- 奇数集对加法**不**封闭： $1 + 1 = 2$ 不是奇数。
- $\{0, 1, 2\}$ 对「加 1」这一元运算**不**封闭，因为 $\varphi(2) = 3 \notin \{0, 1, 2\}$ 。



例 3.9 (工程直觉 (尚不形式化)). 若「已具备的能力集合」 Γ 在「取直接依赖」这一操作下**不**封闭, 就意味着：你以为齐了, 但某个绿结点还缺未列入的先决——这正是后文 Missing 要抓的现象。

3.5.2 闭包：最小的封闭扩张

直观想法：给定种子集合 A , 以及若干「生成规则」(运算), 我们希望找到**既包含 A , 又对规则封闭, 又尽可能小的集合**。这个最小集合就叫做 A 的闭包。

定义 3.16 (闭包 (由封闭性刻画)). 设 U 为全集, $A \subseteq U$, 并固定一组使子集可谈「封闭」的运算规则 $\mathcal{R}$ 。称 $B \subseteq U$ 为 A 关于 $\mathcal{R}$ 的一个**闭包**, 若：

1. $A \subseteq B$ (包含种子);

2. B 对 $\mathcal{R}$ 封闭;
3. 若 $C \supseteq A$ 且 C 对 $\mathcal{R}$ 封闭, 则 $B \subseteq C$ (B 是所有「含 A 的封闭集」中的**最小者**)。

若这样的 B 存在, 常记作 $\text{Cl}_{\mathcal{R}}(A)$, 或在上下文明确时记 $\text{Cl}(A)$ 。

定理 3.2 (闭包的存在性 (有限情形够用版)). 若 U 有限, 则对任意 $A \subseteq U$ 与任意「从已有元素产生新元素」的规则族 $\mathcal{R}$ (形式: 由已有元组确定性地加入有限个新点), $\text{Cl}_{\mathcal{R}}(A)$ 存在且唯一, 并可取为

$$\text{Cl}_{\mathcal{R}}(A) = \bigcap \{C \subseteq U \mid A \subseteq C, C \text{ 对 } \mathcal{R} \text{ 封闭}\}.$$

(任意一族含 A 的封闭集, 其交仍含 A 且封闭; 有限全集上该族非空, 因 U 本身封闭。)

证明梗概. 验证「含 A 的封闭集」对任意交封闭, 故交集是最小元。唯一性由最小性立即得到。□

注 3.5 (两种看法同一个对象). • **由外向内**: 所有合格封闭集的交;

- **由内向外**: 从 A 出发反复施加规则, 直到不能再扩大 (见下一小节迭代)。

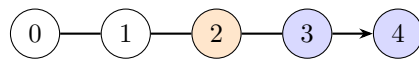
后文算法几乎总用「由内向外」。

定义 3.17 (迭代构造 (一般模板)). 设规则是: 对当前集合 X , 令 $\Phi(X)$ 为「把 X 对规则应用一轮后应有的元素」(要求 $X \subseteq \Phi(X)$)。定义

$$A_0 = A, \quad A_{k+1} = \Phi(A_k) \quad (k \geq 0), \quad A_{\infty} = \bigcup_{k \geq 0} A_k.$$

在有限全集且 Φ 单调时, 存在 K 使 $A_K = A_{K+1} = \cdots = A_{\infty}$, 且 $A_{\infty} = \text{Cl}(A)$ 。

例 3.10 (纯数学: 对「加 1」的闭包). 取 $U = \{0, 1, 2, 3, 4\}$, $\varphi(x) = x + 1$ (约定 $\varphi(4) = 4$ 以免越界), $A = \{2\}$ 。则迭代: $\{2\} \rightarrow \{2, 3\} \rightarrow \{2, 3, 4\} \rightarrow \{2, 3, 4\}$ 。故 $\text{Cl}(A) = \{2, 3, 4\}$ ——包含种子, 并对 φ 封闭的最小集。



种子 闭包多出来的点

3.5.3 闭包算子

不必每次都提运算 $\mathcal{R}$, 也可以直接把「取闭包」看成作用在幂集上的函数 (幂集 2^U 的定义见 §3.2)。

定义 3.18 (闭包算子). 设 U 为集合。映射 $\text{Cl}: 2^U \rightarrow 2^U$ 称为**闭包算子**, 若对任意 $A, B \subseteq U$ (即对任意 $A, B \in 2^U$):

1. **扩张性**: $A \subseteq \text{Cl}(A)$;
2. **单调性**: $A \subseteq B \Rightarrow \text{Cl}(A) \subseteq \text{Cl}(B)$;
3. **幂等性**: $\text{Cl}(\text{Cl}(A)) = \text{Cl}(A)$ 。

称 $\text{Cl}(A)$ 为 A 的**闭包**; 若 $A = \text{Cl}(A)$, 称 A 为**闭集** (对该算子而言)。

命题 3.1 (闭集即「已经封闭」). A 为闭集当且仅当 $\text{Cl}(A) = A$ 。对由规则 $\mathcal{R}$ 导出的闭包算子, 闭集恰好是对 $\mathcal{R}$ 封闭的那些子集。

性质 3.2 (闭包是「补全到封闭」). $\text{Cl}(A)$ 可以理解为: 从 A 出发, 把所有被规则逼出来的点都补上, 直到不能再补。幂等性说: 补第二次不会再增加新点。

例 3.11 (纯数学: 若干熟悉的闭包).

场景	种子 A	闭包在补什么
实数区间	$\{0, 1\}$	对「中间点」封闭 $\rightarrow [0, 1]$ (拓扑闭包的朴素版)
线性代数	若干向量	对加与数乘封闭 $\rightarrow$ 生成子空间
关系	边集 E	对「可走更长路径」封闭 $\rightarrow E^*$ (下一小节)

3.5.4 关系的传递闭包: 闭包的第一个正式例子

现在回到二元关系。这是连接「一般闭包」与「图上依赖闭包」的桥梁。

以下固定有限集 V , 关系 $R \subseteq V \times V$ 。

定义 3.19 (关系的复合与幂). 对 $R, S \subseteq V \times V$, 定义复合

$$R \circ S = \{(u, w) \mid \exists v, (u, v) \in R, (v, w) \in S\}.$$

令 $R^1 = R$, $R^{k+1} = R^k \circ R$, 并令 $R^0 = \Delta_V = \{(v, v) \mid v \in V\}$ 。注意: R^0 是**单独约定**的恒等关系, 表示长度为 0 的途径; 它**不是**把 R 与自身复合得到的 (复合从 $R^1, R^2, \dots$ 才开始)。

定义 3.20 (传递闭包与自反传递闭包).

$$R^+ = \bigcup_{k \geq 1} R^k, \quad R^* = \bigcup_{k \geq 0} R^k = R^0 \cup R^+.$$

称 R^+ 为 R 的**传递闭包**, R^* 为**自反传递闭包**。

命题 3.2 (R^* 确实是闭包). 在「关系集合」 $U = 2^{V \times V}$ 上, 若规则是「若有 $(u, v), (v, w)$ 则加入 (u, w) , 且保留全部恒等对」, 则从 R 生成的闭包恰为 R^* 。特别地: $R \subseteq R^*$, R^* 传递且自反, 且是满足这些性质的最小关系。

定理 3.3 (传递闭包与路径). $(u, v) \in R^k$ 当且仅当存在长度恰为 k 的 R -途径从 u 到 v 。因而 $(u, v) \in R^*$ 当且仅当 u 经若干步 (含 0 步) 到达 v 。

例 3.12 (纯数学: 三点关系的传递闭包). 令 $V = \{a, b, c\}$, $R = \{(a, b), (b, c)\}$ 。逐项按定义展开:

$$R^0 = \Delta_V = \{(a, a), (b, b), (c, c)\} \quad (\text{恒等关系; 不是由 } R \text{ 复合得到}),$$

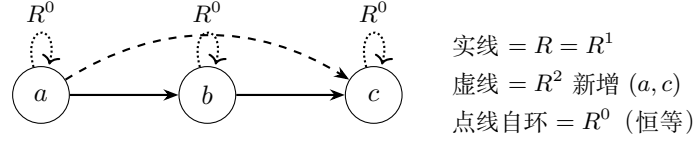
$$R^1 = R = \{(a, b), (b, c)\},$$

$$R^2 = R \circ R = \{(a, c)\} \quad (\text{复合: 经中间点 } b),$$

$$R^+ = R^1 \cup R^2 = \{(a, b), (b, c), (a, c)\} \quad (\text{传递闭包: 只含正长度路径}),$$

$$R^* = R^0 \cup R^+ = \{(a, a), (b, b), (c, c), (a, b), (b, c), (a, c)\}.$$

因此：\$(a, a)\$ 这类对角元来自 \$R^0\$ (0 步路径约定)，**不是** \$R \circ R\$ 算出来的；本例中 \$R\$ 本身也没有自环。图中三种线分别对应三类来源：



若教材图只想强调「传递多出来的边」，可省略自环，但须在式子里保留 R^0 ，以免误以为 \$(a, a)\$ 来自复合。

例 3.13 (工程预告). 若把「\$u\$ 是 \$v\$ 的直接先决」看成关系 \$R\$，则「\$u\$ 是 \$v\$ 的直接或间接先决」就是 \$(u, v) \in R^*\$。下一小节把它改写成顶点集合 \$\text{Dep}^*(T)\$。

3.5.5 有向图上的依赖闭包

从此固定有向图 \$G = (V, E)\$，并取边语义为依赖：

$$u \rightarrow v \text{ 表示 「} v \text{ 依赖 } u\text{」 (} u \text{ 先决, } v \text{ 后继).}$$

与 §2 数据流箭头可能相反；**计算闭包以本节为准**。

定义 3.21 (转置图). $G^\top = (V, E^\top)$, $E^\top = \{(v, u) \mid (u, v) \in E\}$ (箭头反向)。

定义 3.22 (直接前驱 / 直接后继).

$$\text{Pred}_G(v) = N^-(v), \quad \text{Succ}_G(v) = N^+(v).$$

定义 3.23 (祖先与后代). 若 $u \rightsquigarrow_G v$ ，称 u 为 v 的**祖先**， v 为 u 的**后代** (含 $u = v$)。

定义 3.24 (单点依赖闭包). 对目标 $T \in V$ ，定义

$$\text{Dep}_G^*(T) = \{u \in V \mid u \rightsquigarrow_G T\} = \{u \in V \mid (u, T) \in E^*\}.$$

恒有 $T \in \text{Dep}_G^*(T)$ 。略下标时写作 $\text{Dep}^*(T)$ 。

注 3.6 (与一般闭包的对接). 取种子 $\{T\}$ ，规则为「若 v 已在集合中，则把 $\text{Pred}(v)$ 全部加入」。则对该规则的闭包恰等于 $\text{Dep}^*(T)$ 。换言之：**依赖闭包** = 「**对取前驱封闭**」的最小扩张。

命题 3.3 (四种等价说法). 对任意 $u, T \in V$ ，下列等价：

1. $u \in \text{Dep}^*(T)$;
2. $(u, T) \in E^*$;
3. 存在路径 $u \rightarrow \cdots \rightarrow T$ 或 $u = T$;
4. 在 $G^\top$ 中 $T \rightsquigarrow_{G^\top} u$ 。

3.5.6 依赖闭包的迭代、算法与算子性质

定义 3.25 (依赖闭包的迭代序列). 为计算 $\text{Dep}^*(T)$, 我们新引入一串辅助集合, 记作

$$D_0(T), D_1(T), D_2(T), \dots$$

(字母 D 仅表示「Dependency 迭代的第几层近似」, 下标 $k = 0, 1, 2, \dots$ 表示迭代步数; 前文并未使用过该记号, 本定义即其首次出现。)

约定:

$$\begin{aligned} D_0(T) &= \{T\} & (\text{第 } 0 \text{ 步: 只有目标本身}), \\ D_{k+1}(T) &= D_k(T) \cup \bigcup_{v \in D_k(T)} \text{Pred}(v) & (\text{第 } k+1 \text{ 步: 把当前集里每点的直接前驱都并进来}). \end{aligned}$$

直观上: $D_k(T)$ 是「从 T 沿依赖边反向走、最多 k 步能摸到的点」所生成的集合 (含更近的点); 随着 k 增大, 集合只增不减。

这与前面「迭代构造 (一般模板)」中的 A_k 是同一类对象: 取种子 $A = \{T\}$ 、一轮算子 $\Phi(X) = X \cup \bigcup_{v \in X} \text{Pred}(v)$ 时, 恰有 $D_k(T) = A_k$ 。

定理 3.4 (迭代收敛到依赖闭包). 对有限图 G , 序列 $\{D_k(T)\}_{k \geq 0}$ 在有限步内稳定, 且

$$\bigcup_{k \geq 0} D_k(T) = \text{Dep}^*(T).$$

更精确地: 存在 $K \leq |V|$ 使对所有 $k \geq K$ 有 $D_k(T) = \text{Dep}^*(T)$ 。

证明梗概. $D_k(T)$ 单调递增且含于有限集 V , 故稳定。稳定时集合对取 Pred 封闭, 且含 T , 故包含一切能走到 T 的顶点; 反之, 任何通向 T 的长度为 ℓ 的路径上的点在 $D_\ell(T)$ 中出现。□

注 3.7 (与一般闭包模板的关系). 因此「迭代收敛到依赖闭包」就是一般闭包「由内向外」构造在依赖图上的具体化: 做够多步之后, $D_k(T)$ 不再变化, 其稳定值就是 $\text{Dep}^*(T)$ 。

引理 3.2 (反向搜索算法). 在有限图上执行: 从 T 出发, 在 $G^\top$ 中做 *BFS/DFS* (等价于沿 G 的边反向走), 访问到的顶点集合恰为 $\text{Dep}^*(T)$ 。时间复杂度 $O(|V| + |E|)$ (邻接表)。

3.5.7 目标集的闭包、基与缺项

前几小节处理的是单个目标 T 。工程上一次任务往往同时盯住多个目标 (或先把目标拆成若干子目标), 并且手里已有一份「已经获证」的能力清单 Γ 。本小节把 Dep^* 从单点推广到有限集合, 再定义缺项与生成集。

为何需要目标集 若只定义 $\text{Dep}^*(T)$, 则同时要完成 T_1, T_2 时, 需要的先决整体是 $\text{Dep}^*(T_1) \cup \text{Dep}^*(T_2)$ 。把它写成 $\text{Dep}^*({T_1, T_2})$, 可避免每次手写大并集, 也便于后文「闭包表」一行写多个目标。

定义 3.26 (有限目标集). 称非空有限集 $A \subseteq V$ 为一次讨论中的**目标集**。其元素仍是顶点 (能力/任务结点); A 只表示「当前要一起完成的那些目标」。单点情形是特例: $A = \{T\}$ 。

定义 3.27 (目标集的依赖闭包). 对 $A \subseteq V$, 定义

$$\text{Dep}_G^*(A) = \bigcup_{T \in A} \text{Dep}_G^*(T).$$

并约定 $\text{Dep}_G^*(\emptyset) = \emptyset$. 不致混淆时略去下标 G , 写作 $\text{Dep}^*(A)$ 。

命题 3.4 (目标集闭包的等价说法). 对任意 $A \subseteq V$ 与 $u \in V$, 下列等价:

1. $u \in \text{Dep}^*(A)$;
2. 存在某个 $T \in A$, 使 $u \in \text{Dep}^*(T)$ (即 $u \rightsquigarrow T$ 或 $u = T$);
3. 存在 $T \in A$, 使 $(u, T) \in E^*$ 。

因而 $\text{Dep}^*(A)$ 是「能通向 A 中至少一个目标的全部祖先 (含这些目标自身)」。

命题 3.5 (Dep^* 作为幂集上的闭包算子). 定义 $\text{Cl}: 2^V \rightarrow 2^V$ 为 $\text{Cl}(A) = \text{Dep}^*(A)$ (此处 2^V 为 V 的幂集, 见 §3.2)。则 Cl 满足:

1. **扩张**: $A \subseteq \text{Cl}(A)$;
2. **单调**: $A \subseteq B \Rightarrow \text{Cl}(A) \subseteq \text{Cl}(B)$;
3. **幂等**: $\text{Cl}(\text{Cl}(A)) = \text{Cl}(A)$ 。

因此 Cl 是前文意义下的**闭包算子**。进一步: A 对该算子为闭集 (即 $\text{Cl}(A) = A$) 当且仅当 A 对「取直接前驱」封闭——若 $v \in A$ 则 $\text{Pred}(v) \subseteq A$ 。

证明梗概. (扩张) 每个 $T \in A$ 满足 $T \in \text{Dep}^*(T)$, 故 $A \subseteq \text{Dep}^*(A)$ 。

(单调) 并集对包含关系显然单调。

(幂等) 若 $u \in \text{Dep}^*(\text{Dep}^*(A))$, 则 u 能通向 $\text{Dep}^*(A)$ 中某点 v , 而 v 又能通向 A 中某点, 故 u 能通向 A 中某点, 即 $u \in \text{Dep}^*(A)$ 。

闭集刻画: 对取前驱封闭 $\Leftrightarrow$ 含某点则含其直接前驱, 归纳得含全部祖先。 $\square$

性质 3.3 (并与交的粗界). 对任意 $A, B \subseteq V$,

$$\text{Dep}^*(A \cup B) = \text{Dep}^*(A) \cup \text{Dep}^*(B), \quad \text{Dep}^*(A \cap B) \subseteq \text{Dep}^*(A) \cap \text{Dep}^*(B).$$

右式一般**不能**改成等号 (「交的闭包」可以严格小于「闭包的交」)。

已证书集与缺项 闭包回答「理论上需要谁」; 工程上还要问「其中哪些已经有证书、还缺哪些」。

定义 3.28 (已证书集). 固定讨论时刻, 令 $\Gamma \subseteq V$ 表示当前**已获得证书**的顶点集合 (方法论里: 已通过约定验收、允许被引用的能力)。 Γ 随时间可变 (获证则扩大; Dirty 则缩小), 本小节先把它当作给定的集合参数。

定义 3.29 (缺项集). 对目标集 $A \subseteq V$ 与已证书集 $\Gamma \subseteq V$, 定义**缺项集**

$$\text{Missing}_\Gamma(A) = \text{Dep}^*(A) \setminus \Gamma = \{u \in \text{Dep}^*(A) \mid u \notin \Gamma\}.$$

当 $A = \{T\}$ 时简记为 $\text{Missing}_\Gamma(T)$ 。

定义 3.30 (闭包已满). 若 $\text{Missing}_\Gamma(A) = \emptyset$, 称目标集 A 相对 Γ **闭包已满** (亦称「依赖已齐备」)。此时 $\text{Dep}^*(A) \subseteq \Gamma$: 完成 A 所需的全部先决都已在证书集中。

命题 3.6 (缺项的基本性质). 对任意 $A \subseteq V$ 与 $\Gamma, \Gamma' \subseteq V$:

1. $\text{Missing}_\Gamma(A) \subseteq \text{Dep}^*(A)$;
2. $\text{Missing}_\Gamma(A) = \emptyset \iff \text{Dep}^*(A) \subseteq \Gamma$;
3. 若 $\Gamma \subseteq \Gamma'$, 则 $\text{Missing}_{\Gamma'}(A) \subseteq \text{Missing}_\Gamma(A)$ (证书只增时, 缺项只减或不增);
4. $\text{Dep}^*(A)$ 可划分为 $(\text{Dep}^*(A) \cap \Gamma) \dot{\cup} \text{Missing}_\Gamma(A)$ (已证书部分与缺项部分不相交, 并起来恰为闭包)。

注 3.8 (闭包表在算什么). 后文「先交闭包表」在数学上至少要能填写: 目标集 A 、当前 Γ 、算出的 $\text{Dep}^*(A)$, 以及 $\text{Missing}_\Gamma(A)$ 。若缺项非空, 门禁应阻止「直接写顶层实现」, 而要求先对缺项做 Probe/Certify。

生成集与基 (为「最小补洞」作准备) $\text{Dep}^*(T)$ 可能很大; 有时只需指出一堆「种子点」, 其闭包已能覆盖整个 $\text{Dep}^*(T)$ 。

定义 3.31 (生成集). 设 $T \in V$ 。若 $B \subseteq V$ 满足

$$\text{Dep}^*(B) = \text{Dep}^*(T),$$

则称 B 为 $\text{Dep}^*(T)$ 的一个**生成集** (亦称「 B 生成 $\text{Dep}^*(T)$ 」)。特别地, 取 $B = \{T\}$ 时, 由定义 $\text{Dep}^*(\{T\}) = \text{Dep}^*(T)$, 故 $\{T\}$ 总是生成集。

定义 3.32 (极小生成集与基). 若 B 是 $\text{Dep}^*(T)$ 的生成集, 且 B 的任何真子集都不再是生成集, 则称 B 为**极小生成集**。在有限图上极小生成集一定存在 (从任意生成集不断删点直至不能再删)。本教材把「基」暂当作极小生成集的同义语; 更细的唯一性/规范基问题留待门禁章。

定义 3.33 (相对 Γ 的缺项生成集). 工程上更关心: 在已经知道 Γ 的前提下, 还需要新获证的是哪些点。若 $B \subseteq \text{Missing}_\Gamma(T)$ 满足

$$\text{Dep}^*((\text{Dep}^*(T) \cap \Gamma) \cup B) = \text{Dep}^*(T),$$

则称 B 为相对 Γ 的一个**缺项生成集**: 把「已有证书」与「还打算补证的 B 」合在一起, 闭包就能盖住整个 $\text{Dep}^*(T)$ 。最小补洞问题即寻找尽可能小的缺项生成集 (算法细节后置)。

性质 3.4 (菜单、 Γ 与闭包). *INDEX* 等「菜单」提供构造 Γ 的**证据来源**, 但

$$\text{菜单上有许多绿项} \not\Rightarrow \text{Missing}_\Gamma(T) = \emptyset.$$

必须先算 $\text{Dep}^*(T)$, 再与当前 Γ 做差得到缺项。菜单回答「可能有哪些能力」; 闭包与缺项回答「为了 T 必须有谁、还缺谁」。

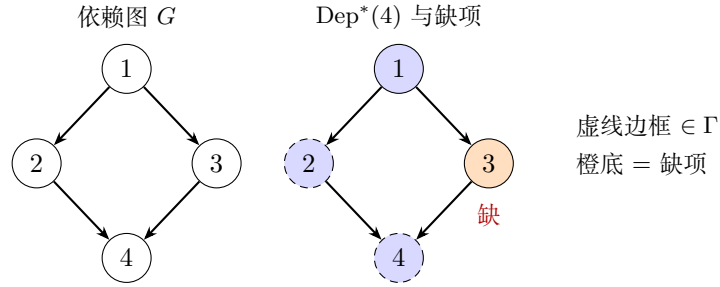
例 3.14 (纯数学：四点图上的闭包与缺项). 续 $V = \{1, 2, 3, 4\}$, $E = \{(1, 2), (1, 3), (2, 4), (3, 4)\}$ (依赖语义：箭头指向依赖者)。则 $\text{Pred}(4) = \{2, 3\}$,

$$D_0(4) = \{4\}, D_1(4) = \{4, 2, 3\}, D_2(4) = \{4, 2, 3, 1\} = \text{Dep}^*(4).$$

取目标集 $A = \{4\}$ 。若 $\Gamma = \{1, 2, 4\}$ ，则

$$\text{Dep}^*(4) \cap \Gamma = \{1, 2, 4\}, \quad \text{Missing}_\Gamma(4) = \{3\}.$$

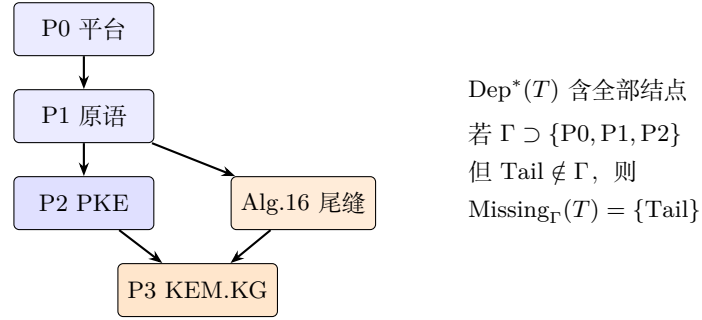
此时 $\{3\}$ 是一个缺项生成集：补证 3 之后闭包已满。又 $\text{Dep}^*(\{2, 3\}) = \text{Dep}^*(4)$ ，故 $\{2, 3\}$ 是 $\text{Dep}^*(4)$ 的一个生成集（且为极小：去掉 2 或 3 任一，闭包都到不了 4 的全部祖先）。



例 3.15 (工程：KEM.KeyGen 的依赖闭包示意). 取依赖语义下的示意 DAG (结点名为能力，非数据)：边表示「箭头终点依赖箭头起点」。令 $T = \text{KEM.KG}$ 。则

$$\text{Dep}^*(T) = \{P0, P1, P2, \text{Tail}, \text{KEM.KG}\}.$$

若当前 Γ 已含 P0–P2，但尚未给尾缝发证，则 $\text{Missing}_\Gamma(T) = \{\text{Tail}\}$ ，目标相对 Γ 未闭包已满。



§2 资产清单 $\approx$ 当时对 $\text{Dep}^*(T)$ 与 Γ 的手工对照；未显式计算 $\text{Missing}_\Gamma(T)$ 就开工，正是「菜单 $\neq$ 说明书」的来源。

3.6 三层对象：库、一次派生、过程

进入形式语言之前，必须把三个层次从定义上分开；否则后文「DAG / 语法树 / 自动机」会混用。

定义 3.34 (能力 DAG (库)). **能力 DAG** 是 DAG $G_{\text{lib}} = (V_{\text{lib}}, E_{\text{lib}})$ ，顶点为可命名、可获证的能力，边为依赖（语义依赖或已声明接缝依赖）。它描述引理库的**静态**结构，不随单次任务进度改写边集（结点证书状态可变，见配置层）。

定义 3.35 (一次任务与派生对象). 固定目标 $T \in V_{\text{lib}}$ 与文法 $\mathcal{G}$ (定义见后文)。一次任务指试图使 T 获证的一次预研作业。该作业中, 从开始符号出发按产生式改写所得的符号串序列称为**派生**; 其树形记录称为**派生树**。派生树是 G_{lib} 上依赖信息的一次使用痕迹, 不是整库拷贝。

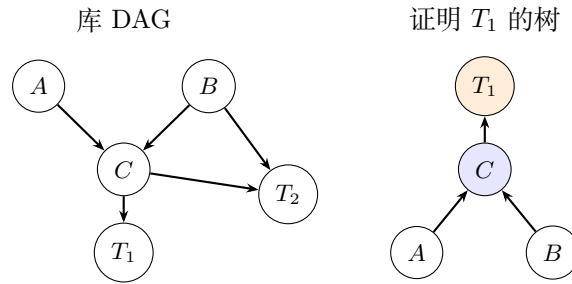
定义 3.36 (配置轨迹). 预研过程中的状态序列 $q_0 \rightarrow q_1 \rightarrow \cdots \rightarrow q_m$, 其中每个 q_i 为配置 (Γ_i, F_i, S_i) (定义见迁移系统一节), 称为**配置轨迹**。它记录证书如何增减、前沿如何消解。

命题 3.7 (三层分工).

对象	回答	不回答
能力 DAG	库中谁依赖谁	某次任务如何逐步展开
派生树	该次任务的展开句子 / 树形	全局共享边的全貌
配置轨迹	证书如何增减; 接受/拒绝	输出是否符合 FIPS 字节

DAG 是库; 派生是一次作业; 轨迹是作业过程。

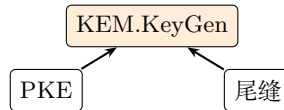
例 3.16 (纯数学: 引理库 vs 一次证明树). 左图: 引理库 DAG (共享)。右图: 证明 T 时实际用到的派生树 (一次作业)。结点 L_2 可被许多证明共享, 但右图只出现本次用到的边。



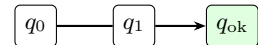
库 (INDEX 依赖直觉)



一次派生 (本次 KeyGen)



配置轨迹 (示意)



Γ 增大 / F 消解

例 3.17 (工程: 三层对照图).

3.7 字母表、字与语言

定义 3.37 (字母表与字). 非空有限集 Σ 称为**字母表**; 其元素称为**字母**。 Σ 上的**字**是有限序列 $w = a_1 a_2 \cdots a_n$ ($n \geq 0, a_i \in \Sigma$); $n = 0$ 时称为**空字**, 记 ε 。称 $n = |w|$ 为字长。全体字之集记 Σ^* (Kleene 星)。又记 $\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$ 。

定义 3.38 (前缀、后缀与因子). 设 $w, x \in \Sigma^*$ 。

- 若存在 y 使 $w = xy$, 称 x 为 w 的**前缀**;
- 若存在 y 使 $w = yx$, 称 x 为 w 的**后缀**;
- 若存在 y, z 使 $w = yxz$, 称 x 为 w 的**因子** (子字)。

定义 3.39 (语言). 若 $L \subseteq \Sigma^*$, 则称 L 为 Σ 上的一个**语言**。称 w 对 L **合法** (或 w 属于 L), 若 $w \in L$ 。

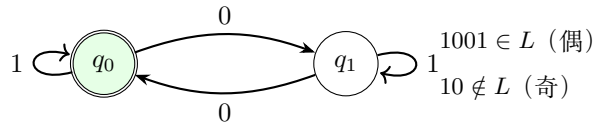
定义 3.40 (语言的连接与星闭包). 对 $x, y \in \Sigma^*$, **连接** xy 表示把 y 接在 x 后。对语言 $L, M \subseteq \Sigma^*$, 定义

$$LM = \{xy \mid x \in L, y \in M\}, \quad L^0 = \{\varepsilon\}, \quad L^{k+1} = L^k L, \quad L^* = \bigcup_{k \geq 0} L^k.$$

性质 3.5 (连接对合法性不必封闭). 一般地, $x \in L$ 且 $y \in L$ **不能**推出 $xy \in L$ 。(这是后文「组合不免费」的纯语言版本。)

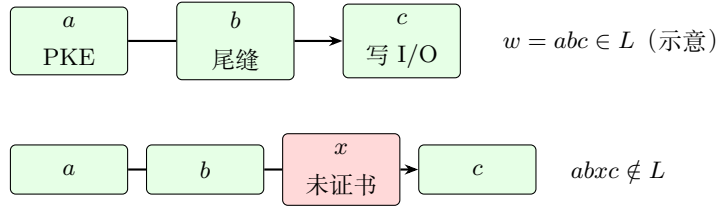
命题 3.8 (Σ^* 的通用性). 任一语言都是 Σ^* 的子集; 讨论「合法计划」即指定某个 $L \subseteq \Sigma^*$ 。门禁的数学内核是: 给定 w , 判定 $w \in L$ 是否成立 (成员判定, 见后文)。

例 3.18 (纯数学: 偶数个 0 的语言(附图)). 取 $\Sigma = \{0, 1\}$, $L_{\text{even}0} = \{w \in \Sigma^* \mid w \text{ 中 } 0 \text{ 的个数为偶数}\}$ 。下图用两状态自动机识别该语言 (q_0 接受): 读 0 翻转奇偶, 读 1 不动。



注意: $0 \notin L$ 但 $00 \in L$, 故不能把「每个字母合法」误当成「连接合法」。

例 3.19 (工程: 实现计划当作字(附图)). 将原子动作编码为字母, 例如 $a = \text{RunPKE}$, $b = \text{Alg16Tail}$, $c = \text{WriteIO}$ 。合法短计划 $w = abc$ 画成动作序列; 插入未证书动作 x 得 $abxc \notin L$ 。



3.8 文法、产生式、派生与派生树

定义 3.41 (上下文无关文法 (本教材用法)). 上下文无关文法是四元组 $\mathcal{G} = (N, \Sigma, P, S)$, 其中:

- N 为有限**非终结**符集;
- Σ 为有限**终结**符集, $N \cap \Sigma = \emptyset$;
- $S \in N$ 为**开始符号**;
- P 为有限**产生式**集, 每个产生式形如 $A \rightarrow \alpha$, 其中 $A \in N$, $\alpha \in (N \cup \Sigma)^*$ 。

定义 3.42 (一步派生与多步派生). 若 $\alpha = \alpha_1 A \alpha_2$, $A \rightarrow \beta \in P$, 则称 α **一步派生**出 $\alpha_1 \beta \alpha_2$, 记 $\alpha \Rightarrow \alpha_1 \beta \alpha_2$ 。 $\Rightarrow^*$ 表示 $\Rightarrow$ 的自反传递闭包 (零步或多步派生)。

定义 3.43 (文法生成的语言).

$$L(\mathcal{G}) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}.$$

定义 3.44 (派生树). 对一次从 S 到 $w \in \Sigma^*$ 的派生, 其**派生树** (语法树) 是满足以下条件的有根有序树: 根标记为 S ; 内部结点标记为非终结符; 叶子从左到右连接恰为 w ; 若内部结点标记 A 且其孩子标记依次拼成 α , 则 $A \rightarrow \alpha \in P$ 。

定理 3.5 (派生与派生树). 对上下文无关文法, $\mathcal{G}$ 能派生出 w 当且仅当存在以 w 为叶子连接的派生树。

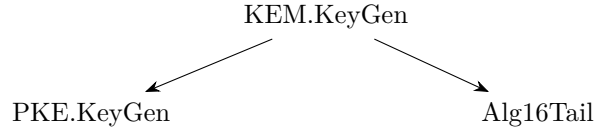
注 3.9 (方法论中的对应). 非终结符 $\approx$ 尚未落到已证书原子步骤的目标; 终结符 $\approx$ 已证书调用或不可再分的合法动作; 产生式 $\approx$ 「目标可展开为哪些子目标 / 接缝」的改写规则。

例 3.20 (纯数学: 匹配括号的文法片段). 令 $N = \{S\}$, $\Sigma = \{(),\}$, 产生式 $S \rightarrow (S) \mid SS \mid \varepsilon$. 则 $() \in L(\mathcal{G})$, $()() \in L(\mathcal{G})$, 而 $() \notin$ 的补——更精确地说 $() \notin L(\mathcal{G})$ 。派生树记录括号如何嵌套, 而不仅仅是字母的前后顺序。

例 3.21 (工程: KEM.KeyGen 的改写). 直觉产生式 (符号仅作说明):

$$\text{KEM.KeyGen} \rightarrow \text{PKE.KeyGen} \cdot \text{Alg16Tail}.$$

若 PKE.KeyGen 已是终结的已证书调用, 而 Alg16Tail 仍是非终结接缝, 则派生树根为 KEM.KeyGen , 一侧叶子为 PKE , 另一侧继续展开尾缝直至获证动作。



3.9 成员判定、禁字与组合不免费

定义 3.45 (成员判定问题). 给定语言 $L \subseteq \Sigma^*$ 与字 $w \in \Sigma^*$, 判断是否 $w \in L$, 称为 L 的**成员判定**。

定义 3.46 (禁字集). 设 $\text{Forb} \subseteq \Sigma^*$. 称语言 L **避开** Forb , 若

$$L \subseteq \{w \in \Sigma^* \mid \text{不存在 } f \in \text{Forb} \text{ 作为 } w \text{ 的因子}\}.$$

其中「因子」指存在 x, y 使 $w = xfy$ 。

定义 3.47 (组合不免费 (方法论公理图式)). 称语言 L 满足**组合不免费**, 若存在 $x, y \in L$ 使得 $xy \notin L$. 在预研建模中我们**不假定** L 对连接封闭; 若工程上需要把 x 与 y 顺序拼装, 通常引入新的非终结符 (接缝) 与产生式, 并要求该接缝获证。

命题 3.9 (菜单不能代替成员判定). 设 $\Sigma_0 \subseteq \Sigma$ 为「菜单上可见的字母」。仅给出 Σ_0 不能唯一确定 L . 尤其, L 还依赖产生式集合与禁字集 Forb 。

例 3.22 (纯数学). 取 $L = \{a, b\} \subseteq \{a, b\}^*$. 则 $a, b \in L$ 但 $ab \notin L$, 故组合不免费。若希望允许 ab , 必须把 ab 显式加入 L , 或增加产生式 (例如 $S \rightarrow ab$)。

例 3.23 (工程). PKE.KeyGen 计划与「某段哈希拼接」各自可能合法, 但「在新的 2-launch 边界下拼接二者」不必自动合法——这正是接缝节点 (如 Alg.16 尾缝、 KemKgTailFused) 存在的理由。
 frozen/ 文书对应 Forb 的工程来源之一。

注 3.10 (门禁雏形). Agent 提交「闭包表 + 展开计划 w 」时, 门禁检查的是: $\text{Missing}_\Gamma(T) = \emptyset$ (全依赖已覆盖) 且存在派生表明 $w \in L(\mathcal{G})$, 并且 w 避开 Forb . 这是「先交闭包表, 再写码」的数学表述。

3.10 迁移系统、配置与接受

定义 3.48 (迁移系统). **迁移系统**是三元组 $\mathcal{T} = (Q, \rightarrow, Q_0)$, 其中 Q 为配置集, $\rightarrow \subseteq Q \times Q$ 为转移关系, $Q_0 \subseteq Q$ 为初始配置集。若 $(q, q') \in \rightarrow$, 记 $q \rightarrow q'$ 。

定义 3.49 (预研配置). **预研配置**是三元组 $q = (\Gamma, F, S)$, 其中

- $\Gamma \subseteq V_{\text{lib}}$ 为已证书能力集;
- F 为前沿目标的有限多重集 (待展开或待获证的目标);
- S 为约束库 (含 I/O 契约、接缝声明、禁字集等)。

定义 3.50 (基本转移 (名称可修订)). 在约束 S 允许的前提下, 定义以下几类转移 (均为 $\rightarrow$ 的子集):

名称	作用
Expand	按产生式展开 F 中某目标, 更新缺项
Probe	对缺项启动探针/预研以寻求证据
Certify	验收通过后将结点写入 Γ
Compose	声明并认证接缝 (落实组合不免费)
Dirty	使部分证书失效 (见下一节)
Freeze	将某路线写入禁字 / 否定约束

定义 3.51 (接受与拒绝). 固定目标 T 与生产 I/O 契约 $\mathcal{C}$ 。

- 配置 $q = (\Gamma, F, S)$ 称为**接受配置**, 若 $T \in \Gamma$ 、 F 为空 (或仅含已放电目标), 且 $\mathcal{C}$ 满足;
- 若转移过程试图使用 Forb 中的因子、或引用 $\notin \Gamma$ 的依赖边且未先 $\text{Probe}/\text{Certify}$, 则进入**拒绝** (具体可建模为沉入拒绝阱状态)。

定义 3.52 (合法性与幻觉 (工作定义)). 1. 称一次预研过程**合法**, 若它对应迁移系统中从某初始配置到接受配置的有限转移序列, 且每一步都符合产生式与 S 中约束。

2. 称行为构成**幻觉**, 若它执行了非法转移, 或在图上使用了无证书边 (等价地: 计划字 $w \notin L$ 或未避开 Forb)。

语义 oracle (FIPS / liboqs / golden) 判定「计算结果是否正确」; 本定义判定「砖是否允许那样砌」。

例 3.24 (纯数学: 门禁自动机小品). 状态 $\{q_0, q_{\text{ok}}, q_{\text{bad}}\}$, 字母 $\{a, b\}$ 。 $q_0 \xrightarrow{a} q_0$, $q_0 \xrightarrow{b} q_{\text{ok}}$, 其余进 q_{bad} 。则字 aab 被接受, ba 被拒绝。这与「先合法前缀、再结束符」的门禁同构。

例 3.25 (工程). 初始 Γ 含 stable PKE 等; $F = \{\text{kem.keygen}\}$ 。经 Expand 得到尾缝缺项 $\rightarrow \text{Compose}/\text{Probe} \rightarrow \text{Certify}$ 后 $T \in \Gamma$, 到达接受。若中途从 `frozen-exp-*` 抄实现, 则触发拒绝 (Freeze 已写入 Forb)。

3.11 证书的非单调更新

定义 3.53 (单调与非单调证书库). 称证书更新机制为**单调**的, 若任意转移满足 $\Gamma \subseteq \Gamma'$ (证书集只增不减). 若允许某次转移后 $\Gamma' \subsetneq \Gamma$, 则称为**非单调**的。

定义 3.54 (脏标记). 对 $v \in \Gamma$, 若因上游接口、布局或同步约定变更, 使 v 的既有证书不再被承认, 则称 v 被标记为 **dirty**, 并在转移 **Dirty** 下将 v 移出 Γ (或移入「待重证」区)。具体存储格式后文工程化。

命题 3.10 (工程引理库非单调). 本方法论采用的证书库是非单调的: 存在合法的 **Dirty** 转移使 Γ 严格缩小。

推论 3.2 (过期证书不可当作永真公理). 门禁在引用 $v \in \Gamma$ 时, 须能够回答「自获证以来上游是否触发 **Dirty**」。

例 3.26 (纯数学: 撤回引理). 证明系统中若发现引理 L_1 的证明有漏洞而撤回, 则所有依赖 L_1 的定理证书失效。这与「断言永真后只增不减」的理想图景不同。

例 3.27 (工程). P1 某能力的 tiling/同步壳变更后, 曾基于旧 iface 的 P2/P3 证书应标 **dirty**, 不得因「上次 PASS」而默认继续合法。

3.12 回扣第 2 章四组问题

问题组	用本章编号对象重述
第一组	语义 $\text{oracle} \neq L(\mathcal{G})$; 合法砌法由 DAG、产生式与 Forb 给出。
第二组	组合不免费: 存在 $x, y \in L$ 使 $xy \notin L$; 接缝靠 Compose 获证。
第三组	伪代码 + 菜单至多给出 Σ 的线索, 缺少成员判定则搜索空间为 Σ^* 量级。
第四组	成功过程应沉淀 $\text{Dep}^*(T)$ 、派生痕迹与 Γ 更新; 上游变更走 Dirty 。

3.13 本章小结与下一章预告

本章按教材体例建立了方法论数学底座, 主链为:

关系/函数 $\rightarrow$ 有向图 $\rightarrow$ DAG/拓扑序 $\rightarrow$ 一般闭包 $\rightarrow$ 传递闭包 $\rightarrow$ 依赖闭包 $\rightarrow$ 语言与文法 $\rightarrow$ 迁移系统

每个关键定义均配有**纯数学例**与**工程例**。刻意未完成的部分: 产生式的仓库级形式化语法、闭包表文件格式、可判定性证明——留给后续章节迭代。

下一章 (第 4 章) 把上述对象**填写成工程字段**: 节点属性、边的三类、证书状态命名等。

4 从数学到工程对象：节点、边与证书

上一章给了一般语言; 本章把它**具象化**成预研图上的可填写字段。若与第 3 章用语冲突, 以「工程可执行」优先, 并回写修订记录。

4.1 三个层面（复习）

概念	回答的问题	不回答的问题
能力 DAG	引理库里谁依赖谁？	某次任务如何一步步展开？
语法树 / 派生	这次任务的展开句子是什么？	全局所有共享边的全貌？
配置自动机	证书如何增减？何时接受/拒绝？	代数是否符合 FIPS？

4.2 节点（已验证能力）

节点 v 建议携带：

- `id`：稳定名（如 `pke.keygen.k4`）；
- `layer`：P0–P3（方法论分层，不等于目录树一一对应）；
- `iface`：I/O、dtype、形状/布局、同步约定——**可复用的是 `iface`，不是源码**；
- `cert`：路径 + CPU+SIM（+KAT）+ 版本锚点；
- `forbid`：否定约束；
- `repo`：权威目录锚点。

分层语言（P0–P3）

层	含义（本仓库语境）
P3	顶层算子 / 算法入口（如 <code>KEM.KeyGen</code> 、 <code>KEM.Encaps</code> ）
P2	算法构件（如 <code>PKE.KeyGen</code> / <code>Encrypt</code> / <code>Decrypt</code> 全链）
P1	领域原语（ <code>NTT</code> 、 <code>basemul</code> 、 <code>CBD</code> 、 <code>ByteEncode</code> 、 <code>SHAKE</code> ...）—— 作能力名 ，本章不讲其代数
P0	平台公理（ <code>DataCopy</code> 、向量算子、 <code>MMAD+tiling</code> 、 <code>TPipe/TQue</code> 同步壳...）

4.3 边的三种类型

1. **语义依赖**： u 的正确性需要 v 的 `iface`（如「`NTT` 能力」 $\rightarrow$ `PKE.KeyGen`）；
2. **接缝（compose）**： A 、 B 各自有证， $A \circ B$ **仍须**单独成节点获证（「组合不免费」）；
3. **否定边**：显式禁止（`frozen` 模板、未登记计算块、`Host` 默认密码路径等）。

4.4 证书状态（含非单调）

$\text{unproven} \rightarrow \text{probe} \rightarrow \text{lemma} \rightarrow \text{deliverable}$ ， 以及 `dirty`。

`dirty` 表示上游 `iface/tiling` 变更后下游证书失效——引理库 Γ **可收缩**。

4.5 仓库机制与数学对象的对照（直觉）

- 活跃 INDEX.md：当前可引用的能力菜单（ Σ 的工程投影，**还不是** L ）；
- frozen/ + FROZEN.md：否定边；
- baseline-registry：golden 允许调用的已验证计算块；
- customspec：examples 写码的规格门禁（派生计划的文书形态之一）。

4.6 立论（承接第 3 章）

预研开发的合法性 $\approx$ 在已证书能力生成的语言 L 上，由配置自动机搜索接受路径；

幻觉 $\approx$ 非法转移 / 无证书边。

语义 oracle (FIPS/liboqs/golden) 管「算对」；形式方法管「从哪砌砖」。

5 门禁与工作流：Agent 被允许做什么

5.1 合法性规则（v0）

接到目标 T 时，只允许：

1. 展开依赖闭包（语义边 + 接缝边）；
2. 分类：已有 lemma/deliverable；缺项；仅否定边；
3. 缺项 $\Rightarrow$ 先探针/exp 并获证，再引用；禁止在 T 任务里顺便发明 P0/P1；
4. 新接缝 = 新节点，须认证；
5. 验收看 I/O 与 golden/KAT，不看「与参考源码同构」；
6. 自包含：挂 iface，不跨目录偷源码（仓库既有例外除外）；图边 $\neq$ #include。

5.2 四步工作流

1. 目标声明 T、I/O、验收级别 (probe / lemma / deliverable)
2. 闭包展开 P0--P3；ok / missing / dirty；接缝清单
3. 最小补洞 只补 missing 与新接缝；forbid 写入否定边
4. 闭环写回 新节点入图；失败则改规则或 Freeze，不默认可复用

先交闭包表，再写码。customspec / STATUS / registry 是文书形态，不是替代闭包表。

5.3 域无关性

抽出密码词之后，对象仍是：终端 (axiom)、非终端 (goal)、产生式 (compose)、证书 (judgment)。同一套问题适用于非密码 AscendC 预研；ML-KEM 只是 oracle 清晰、接缝丰富的压力测试床。

6 复盘：理论如何对照 *kem.keygen* 与 *kem.encaps*

6.1 复盘的目的

不是证明「我们早就在按理论做」（事后贴标签无价值），而是：

1. 检查分层与接缝是否可被理论语言说清；
2. 找出偏离（例如曾依赖 correctness vendor / frozen 模板）并记为**方法论债**或**否定边**；
3. 总结「数学原理如何被具象化为目录、launch、证书」。

6.2 *kem.keygen* 校准问题单

- P3 是否只依赖已证书 P2，而非在 KeyGen 任务内重写 NTT？
- 2-launch、Alg.16 尾、dk 展开等接缝是否显式？
- baseline-registry 中的 golden 块是否都在图上有 lemma？
- 否定边是否写入（Host 默认禁密码、禁把 liboqs 当 AscendC 规格）？

仓库锚点(查阅用): `examples/stable/stable-fips203-mlkem-kem-keygen-k4/`, `docs/specs/fips203-mlkem1` 及相关 `pass-fix-*` device 探针。

6.3 *kem.encaps* 复盘要点（讨论共识）

- 合法路径应挂已证 `pke.encrypt` 与哈希/G 等 lemma，而不是以 correctness 内 vendor 为实现模板；
- 与 stable Encrypt 的编译期/接缝关系必须写进闭包，而不是「觉得复用了」；
- 若实现史存在混乱探索，复盘文应诚实记录——那是对照组与方法论的动机，不是羞耻。

6.4 期望产出的「具象化对照表」（后续填空）

理论对象	工程具象	证书形态
节点 v	探针/exp/stable 目录	STATUS + run.sh 对拍
语义边	「必须先有某某能力」	INDEX / customspec 引用
接缝节点	多 launch / 融合边界	集成探针单独 PASS
否定边	frozen / forbid 条文	FROZEN.md、Rule
字 $w \in L$	一次任务的闭包表 + 计划	评审通过再写码

7 前瞻：用理论指导尚未完成的 *kem.decaps*

7.1 为什么 Decaps 适合做试金石

Decaps 同时依赖 Encrypt、Decrypt、KeyGen I/O 与 FO 失败路径，是典型的**多父 DAG**；且若理论只能解释已完成故事、不能生成待做清单，则方法论停在修辞。

7.2 前瞻时 Agent（或人类）应交付的最小闭包

1. 目标 I/O 与验收级别；
2. 依赖节点清单（标 ok / missing / dirty）；
3. 接缝清单（含对称失败 / 重加密比对等是否独立成节点）；
4. 否定边（不可引用的 correctness vendor / frozen 实现）；
5. 最小补洞顺序（先探针哪条缝）。

7.3 成功标准（阶段性）

- **弱成功**：闭包表可执行；偏差归因到缺边/假边/脏证书，并回写本章理论；
- **强成功**：在门禁下完成 device/stable 级实现，且过程指标优于 correctness 探索史（见下一章比较轴）。

8 对照组：correctness 标本的意义

8.1 为何保留 correctness

-correctness- 在完全不考虑优化的前提下，只追求可执行性与结果正确：多 kernel launch、多搬运与同步——这是「一定能搭出正确积木」的保守拼法，性能不可用，也几乎吃不进后续 P0/P1 工程优化。

它还记录了一段真实的 Agent 过程史：故意不给确定性拼装路线时，探索成本极高（失败、卡死、回退；曾短期内耗尽大量 token），最终仅得到「结果正确」的代码。

8.2 在方法论中的角色

角色	含义
正确性锚	下界存在性：积木可搭、I/O 可对
结构反例	展示「只求对」会长成怎样的 launch/同步膨胀
过程对照	无确定路线的搜索成本
实验基线	与方法论 Agent、人工指导路径三方比较

纪律：correctness 是**基线标本**，不是活跃实现模板；可读、可对拍、可对比，**禁止**当抄码来源（与 frozen 出门不带码一致）。

8.3 建议比较轴

1. **过程**：token / 轮次 / 回退；是否先交闭包表；非法边是否被拦截；
2. **结构**：launch 数、同步点、lemma 复用、接缝是否显式；
3. **结果**：CPU+SIM；SIM tick 相对 correctness 的倍数。

三档：A 旧式无路线探索 $\sim$ correctness；B 新方法论 Agent；C 人工指导 device/stable。B 不必一次打赢 C；明显优于 A，即方法论第一关。

9 未决议事项、日程与维护

9.1 刻意未决议

- 节点是否 1:1 对应目录名；图的存储是 Markdown 表还是 YAML/JSON；
- 主叙事选工作流网 / Petri 网 / 属性文法 / 判断式中的哪一种；
- 何时写入 Rule/Skill；可判定性与复杂度是否单开理论线。

9.2 近期填空顺序（与「特殊 $\rightarrow$ 一般」一致）

1. 按复盘反馈修订第 3 章（定义精度、例子密度）；把 KeyGen 合法派生轨迹写成一页纸；
2. 第 4 章字段表与仓库文书（闭包表 schema）对齐；
3. Encaps 理论轨迹 vs 实现轨迹对照表；
4. Decaps 前瞻闭包；
5. correctness 对照实验写成可执行协议；
6. 结论稳定后再考虑迁 docs/notes/。

9.3 文档与分支约定

- 本文件为专题**唯一** TeX/PDF 工作副本（无日期后缀），持续修订；
- 讨论纪要：形式语言 DAG 预研讨论纪要.md（同样单文件刷新）；
- 工作分支：research/formal-lang-dag；未经用户明确指令**不合人** main；
- 不记录题外的知识产权保护策略讨论。

9.4 编译

```
bash scripts/xelatex-clean.sh \
  docs/research/从已验证能力到合法派生-面向Agent预研的形式方法教材草案.tex
```